GeoClaw-claude

Skill 编写规范与安全指南

Skill Writing Specification & Security Guidelines

版本：v2.4.0
UrbanComp Lab · 中国地质大学（武汉）
https://urbancomp.net
2025 年 3 月

1 概述

1.1 什么是 Skill

Skill 是 GeoClaw-claude 的核心扩展机制——一个符合统一规范的 Python 脚本文件。每个 Skill 封装一个完整的地理空间分析工作流，可由用户通过命令行、Python API 或自然语言指令调用。

Skill 系统的设计目标是「低门槛、高可复用、安全可信」：研究人员只需掌握基础 Python，即可在 15 分钟内完成一个生产可用的 Skill；SkillManager 负责注册、版本管理和生命周期控制；SkillAuditor 在安装前自动进行安全扫描，阻断恶意或高危代码注入。

1.2 两类典型 Skill 模式

维度	AI 驱动版（requires_ai: True）
分析核心	LLM 综合推理，动态分配权重
依赖 API Key	是（Anthropic / Gemini / OpenAI）
结果可复现	否（LLM 具随机性）
适用场景	需要结合商业逻辑、非结构化信息
示例 Skill	retail_site_ai（商场选址 AI 版）

维度	纯算法版（requires_ai: False）
分析核心	确定性数学算法（MCDA、KDE 等）
依赖 API Key	否，可离线运行
结果可复现	是，相同输入必然相同输出
适用场景	批量筛查、对照实验、需审计的决策
示例 Skill	retail_site_algo（商场选址算法版）

1.3 Skill 生命周期

```
# 1. 创建模板
geoclaw-claude skill new my_skill

# 2. 编写分析逻辑（编辑生成的 .py 文件）

# 3. 安全审计（不安装，仅扫描）
geoclaw-claude skill audit ./my_skill.py

# 4. 安装（自动触发安全审计 + 交互确认）
geoclaw-claude skill install ./my_skill.py

# 5. 运行
geoclaw-claude skill run my_skill --data input.geojson
```

2 Skill 文件结构规范

2.1 完整文件模板

一个合规的 Skill 文件由以下三部分组成：

```
"""
my_skill.py — Skill 名称（简明，无空格）
====================
Skill 功能的一句话描述。

详细说明（可多行）：
  - 算法原理
  - 输入数据要求
  - 输出格式说明

依赖:
  pip install geopandas scipy

示例:
  geoclaw-claude skill run my_skill --data input.geojson
"""
# Copyright (c) 2025 YourName — MIT License

# ── Part 1: SKILL_META（必填元数据）──────────────────────
SKILL_META = {
    "name":        "my_skill",        # 唯一标识符，小写下划线
    "version":     "1.0.0",           # 语义版本号
    "author":      "Your Name",       # 作者姓名/机构
    "description": "功能描述（50字以内）",
    "tags":        ["主题标签"],      # 可选，用于搜索
    "inputs": [
        {"name": "input",  "type": "GeoLayer", "desc": "必填输入图层"},
        {"name": "radius", "type": "float",    "desc": "半径(m)",
"default": 1000},
    ],
    "outputs": [
        {"name": "result", "type": "GeoLayer", "desc": "分析结果图层"},
        {"name": "report", "type": "str",      "desc": "文字报告"},
    ],
    "requires_ai": False,             # 是否需要 LLM API
    "example":     "geoclaw-claude skill run my_skill --data x.geojson",
}

# ── Part 2: 辅助函数（可选）──────────────────────────────
def _helper(data):
    """私有辅助函数，以 _ 开头。"""
    return data

# ── Part 3: run() 入口（必填）────────────────────────────
def run(ctx):
    """Skill 主函数，接收 SkillContext 对象。"""
    # 1. 读取输入
    layer  = ctx.get_layer("input")
    radius = float(ctx.param("radius", 1000))

    # 2. 执行分析
```

```
    from geoclaw_claude.analysis.spatial_ops import buffer
    result = buffer(layer, radius)

    # 3. （可选）AI 分析
    comment = ctx.ask_ai("请分析这些数据的空间分布规律。")

    # 4. 返回结果（自动保存 GeoLayer 到 output_dir）
    return ctx.result(result=result, report=comment)
```

3 SKILL_META 字段规范

3.1 必填字段

字段名	类型	说明与约束
name	str	全局唯一标识符；小写字母+下划线；不能与内置 Skill 同名
version	str	遵循语义版本（SemVer）：MAJOR.MINOR.PATCH
author	str	作者姓名或机构名；用于审计溯源
description	str	功能摘要，建议 20~80 字；显示在 skill list 中

3.2 输入/输出声明（强烈推荐）

inputs / outputs 字段描述 Skill 的接口契约，SkillManager 据此验证参数合法性。每个条目是一个字典：

```
{"name": "layer_name",    # 参数名，与 ctx.get_layer()/ctx.param() 对应
 "type": "GeoLayer",      # 类型: GeoLayer / float / int / str / bool
 "desc": "参数说明",
 "default": None,         # 可选，有默认值则该参数非必填
 "required": True}        # 可选，默认 True；False = 可选参数
```

3.3 完整字段速查表

字段名	是否必填	说明
name	✅ 必填	唯一名称，小写下划线
version	✅ 必填	SemVer 版本号
author	✅ 必填	作者/机构
description	✅ 必填	功能摘要
inputs	推荐	输入参数声明列表
outputs	推荐	输出结果声明列表
tags	可选	主题标签，便于搜索
requires_ai	可选	是否需要 LLM API，默认 False
dependencies	可选	第三方 pip 包列表

example	推荐	完整的命令行调用示例

4 SkillContext API 参考

run(ctx) 接收的 ctx 对象提供以下接口：

4.1 图层访问

```
# 获取已加载图层（默认名为 'input'）
layer = ctx.get_layer('input')      # → GeoLayer
layer = ctx.get_layer('hospitals') # → 按名称获取

# 从文件加载并注册新图层
layer = ctx.load_layer('/path/to/data.geojson', name='pois')
```

4.2 参数读取

```
# 读取参数（支持 CLI 传入的 --key=value 形式）
radius = float(ctx.param('radius_km', 1.5))   # 带默认值
top_n  = int(ctx.param('top_n', 3))
name   = ctx.param('city', 'Wuhan')            # 字符串参数

# 设置参数（在 Skill 内部传参使用）
ctx.set_param('radius_km', 2.0)
```

4.3 AI 分析调用

```
# 基本调用
text = ctx.ask_ai('请分析这些点位的空间聚集特征。')

# 附带数据背景
text = ctx.ask_ai(
    '请评估以下选址方案的优劣：',
    context_data=str(stats_dict)
)

# 注意：未启用 AI（无 --ai 参数）时返回提示字符串
# 使用前判断：if text.startswith('(AI'): ...
```

4.4 结果输出

```
# 所有 GeoLayer 自动保存为 output_dir/<name>.geojson
return ctx.result(
    result=scored_layer,  # GeoLayer → 保存为 result.geojson
    report=report_text,   # str → 不自动保存，由 CLI 打印
)

# 查询输出目录
print(ctx.output_dir)  # → ~/.geoclaw_output 或 --output 指定路径
```

5 Skill 安全规范

5.1 SkillAuditor 概述

GeoClaw-claude 在每次 skill install 前自动运行 SkillAuditor——一个基于静态分析（AST + 正则）的安全扫描器，无需执行代码即可识别潜在恶意行为。

审计分为五个风险等级，安装流程根据等级采取不同响应策略：

等级	分值权重	处理策略
CRITICAL（严重）	+30 分/项	强制要求用户输入 'yes' 确认；默认拒绝
HIGH（高危）	+15 分/项	警告提示，要求用户明确确认
MEDIUM（中等）	+5 分/项	提示后继续安装
LOW（低）	+2 分/项	仅提示，自动继续
INFO（信息）	0 分/项	提示信息，不影响安装

5.2 CRITICAL 级禁止行为

⛔ **禁止** 以下行为会触发 CRITICAL 级警报，安装将被默认阻断。

- 命令执行：os.system()、subprocess.*()、os.popen()
- 代码注入：eval()、exec()、__import__()、compile()+exec
- 代码混淆：base64.b64decode() 用于隐藏 payload
- 数据外泄：requests.post(data=...)、socket.connect()、paramiko/ftplib/smtplib
- 权限提升：ctypes 模块、修改文件执行权限
- 危险导入：import pty（伪终端）

5.3 HIGH 级警告行为

⚠️ **警告** 以下行为会触发 HIGH 级警告，需用户明确确认方可安装。

- 危险文件操作：os.remove()、os.unlink()、shutil.rmtree()
- 文件写入：open(..., 'w') 若路径超出 ctx.output_dir 范围
- 进程操作：os.kill()、os.fork()、signal 模块
- 反序列化：pickle.load()（可执行任意代码）
- 危险模块：pdb、code、marshal、cffi、winreg

5.4 安全编写最佳实践

✅ **推荐** 遵循以下原则可确保 Skill 顺利通过安全审计。

1. 所有文件读写严格限制在 ctx.output_dir 路径内，不操作系统路径。
2. 网络请求仅允许向已知公开数据源（如 OSM Overpass API、天地图）发起 GET 请求。

3. 不读取与分析无关的环境变量；必须读取时在 SKILL_META 中声明原因。
4. 第三方依赖在 SKILL_META.dependencies 中明确列出，避免隐式安装。
5. 不使用 pickle；序列化需求改用 json 或 GeoJSON 格式。
6. 不包含任何混淆代码（base64 编码的可执行内容）。
7. 发布到社区前，先用 geoclaw-claude skill audit 自测，确认无 CRITICAL/HIGH 项。

5.5 安全审计命令

```
# 仅审计，不安装
geoclaw-claude skill audit ./my_skill.py

# 安装时自动审计（默认行为）
geoclaw-claude skill install ./my_skill.py

# 跳过审计（仅限内置 Skill，不推荐用于第三方）
geoclaw-claude skill install ./my_skill.py --no-audit

# Python API 集成
from geoclaw_claude.skill_auditor import SkillAuditor
result = SkillAuditor().audit('./my_skill.py')
print(result.risk_score, result.max_level)
```

6 案例：商场选址分析（两种实现对比）

以「商场选址综合评估」为例，演示 AI 驱动版与纯算法版的设计差异。两个版本已内置于 GeoClaw-claude，可直接使用。

6.1 AI 驱动版（retail_site_ai）

特性	说明
Skill 名称	retail_site_ai
核心引擎	LLM（Claude / Gemini / GPT）
分析方式	Python 计算空间指标 → 打包交给 LLM 综合推理
权重	LLM 动态分配，可结合品牌策略等非结构化信息
requires_ai	True（需配置 API Key）
输出	评分图层 + LLM 撰写的中文分析报告

```
# 运行示例（需配置 API Key）
geoclaw-claude skill run retail_site_ai \
    --data candidates.geojson \
    --ai \
    --radius_km=2.0 \
    --top_n=3
```

6.2 纯算法版（retail_site_algo）

特性	说明
Skill 名称	retail_site_algo
核心引擎	MCDA 多准则决策分析（Min-Max 归一化）
分析方式	四维评分：人口密度、竞争回避、空间分散、交通可达
权重	用户可通过参数指定（--w_pop / --w_comp 等）
requires_ai	False（完全离线可用）
输出	评分图层（含 score_total/rank 列）+ 结构化报告

```
# 运行示例（无需 API Key）
geoclaw-claude skill run retail_site_algo \
    --data candidates.geojson \
    --radius_km=2.0 \
    --top_n=3 \
    --w_pop=0.4 --w_comp=0.3 --w_disp=0.2 --w_road=0.1
```

6.3 如何选择

> 💡 **选择建议** 结果需要面向非专业人员解读、涉及非结构化商业判断 → 用 AI 版；需要可审计、可复现、大批量自动化 → 用算法版；两者可配合使用：先用算法版筛选，再用 AI 版对 Top 候选深度研判。

7 测试规范

7.1 测试文件命名

- Skill 功能测试：tests/test_skills_and_security.py
- 测试用 ID 格式：S01~Sxx（Skill 功能）、A01~Axx（安全审计）

7.2 每个 Skill 必须测试

8. 基本运行不报错，返回预期输出类型
9. 输出图层包含所有声明的字段
10. 数值结果在合理范围内（如分数在 0~100）
11. 边界情况：单候选点、无可选图层、极端参数
12. 无 AI 模式下的回退行为

7.3 安全测试

malicious_skills/ 目录提供三个模拟高危 Skill，用于验证安全审计系统：

文件	主要触发规则	预期结果

evil_exfil.py	os.system + requests.post + os.environ	CRITICAL × 3，passed=False
evil_inject.py	eval + exec + base64 + __import__	CRITICAL × 4，passed=False
evil_file_ops.py	os.remove + shutil.rmtree + subprocess	CRITICAL/HIGH，score ≥ 30

```
# 运行完整安全测试
pytest tests/test_skills_and_security.py -v
# 期望输出: 40 passed
```

8 Skill 发布与社区贡献

8.1 发布清单

13. 通过 geoclaw-claude skill audit 审计，无 CRITICAL/HIGH 项
14. 在 tests/ 中提供至少 5 个测试用例，全部通过
15. SKILL_META 填写完整（含 tags / example / dependencies）
16. 文件头包含完整的 docstring（功能、参数、示例）
17. 附 README 或在 docs/ 中提交使用说明文档
18. 遵循 MIT 或 Apache-2.0 等开源许可证

8.2 提交方式

- 将 Skill .py 文件提交至 PR：geoclaw_claude/skills/community/
- 在 PR 描述中附上 skill audit 输出截图
- 内置 Skill 提交至 skills/builtin/，需由 UrbanComp Lab 审核

📬 **联系** UrbanComp Lab · urbancomp.net · GitHub: github.com/whuyao/GeoClaw_Claude